Question 1: What is K-Nearest Neighbors (KNN) and how does it work in both classification and regression problems?
Ans:- **K-Nearest Neighbors (KNN)** is a **supervised machine learning algorithm** used for both **classification** and **regression**. It is a **non-parametric** and **instance-based (lazy learning)** algorithm because it does not build a model during training. Instead, it stores the training data and makes predictions based on the nearest data points.

How KNN Works

1. Store all the training data.
2. Choose the number of neighbors (**K**).
3. Calculate the distance between the new data point and all training points (commonly using Euclidean distance).
4. Select the **K nearest neighbors**.
5. Make a prediction based on the task:
    - **Classification:** Predict the class that appears most frequently among the K neighbors (majority voting).
    - **Regression:** Predict the average (or weighted average) of the target values of the K neighbors.

KNN for Classification

- Used when the target variable is **categorical** (e.g., Spam/Not Spam, Yes/No).
- The algorithm assigns the class that is most common among the K nearest neighbors.

Example:

- K = 5
- Neighbor classes: **A, A, B, A, B**
- **Prediction:** Class **A** (majority vote).

KNN for Regression

- Used when the target variable is **continuous** (e.g., house price, temperature).
- The algorithm predicts the output by taking the **average** of the K nearest neighbors' values.

Example:

- K = 3
- Neighbor values: **50, 55, 60**
- **Prediction:** (50 + 55 + 60) / 3 = **55**

Advantages

- Simple and easy to understand.
- No training phase (lazy learner).
- Works well for small datasets.
- Can solve both classification and regression problems.

Disadvantages

- Slow for large datasets because it calculates distances to all training points.
- Sensitive to irrelevant features and noisy data.
- Performance depends on the choice of **K**.
- Requires feature scaling because distance calculations are affected by different feature ranges.

Question 2: What is the Curse of Dimensionality and how does it affect KNN performance?
Ans:- The **Curse of Dimensionality** refers to the problems that arise when the number of features (dimensions) in a dataset becomes very large. As the number of dimensions increases, the data points become sparse, and distance-based algorithms like **K-Nearest Neighbors (KNN)** become less effective.

How it Affects KNN Performance

1. **Distance becomes less meaningful**
    - In high-dimensional data, the distances between data points become very similar.
    - It becomes difficult for KNN to identify the true nearest neighbors.
2. **Lower prediction accuracy**
    - Since neighbors may not be genuinely similar, the model makes less accurate predictions.
3. **Higher computational cost**
    - KNN must calculate the distance between the query point and every training sample.
    - More features increase the computation time and memory usage.
4. **Risk of overfitting**
    - High-dimensional data often contains irrelevant or noisy features.
    - KNN may consider these unnecessary features, reducing its generalization performance.

Example

Suppose you are classifying customers:

- With **2 features** (Age and Income), similar customers are easy to identify.
- With **100 features**, many irrelevant features make almost all customers appear equally distant, making it difficult for KNN to find meaningful neighbors.

Ways to Reduce the Curse of Dimensionality

- **Feature Selection:** Keep only the most important features.
- **Dimensionality Reduction:** Use techniques like **Principal Component Analysis (PCA)** to reduce the number of features.

- **Feature Scaling:** Normalize or standardize features before applying KNN.
- **Collect More Data:** More training samples can help represent the high-dimensional space better.

Question 3: What is Principal Component Analysis (PCA)? How is it different from feature selection?
Ans:- **Principal Component Analysis (PCA)** is an **unsupervised dimensionality reduction** technique used to reduce the number of features in a dataset while preserving as much information (variance) as possible. It transforms the original correlated features into a new set of **uncorrelated variables**, called **principal components**.

How PCA Works

1. Standardize the data (if features have different scales).
2. Compute the covariance matrix.
3. Calculate the eigenvalues and eigenvectors.
4. Select the principal components with the highest eigenvalues.
5. Transform the original data into a lower-dimensional space using these components.

The first principal component captures the maximum variance, the second captures the next highest variance (while being orthogonal to the first), and so on.

Feature Selection vs. PCA

Feature Selection	Principal Component Analysis (PCA)
Selects a subset of the original features.	Creates new features called principal components.

Original features remain unchanged.	Original features are transformed into combinations of features.
Maintains feature interpretability.	Principal components are harder to interpret.
Removes irrelevant or redundant features.	Reduces dimensionality by preserving maximum variance.
Can be supervised or unsupervised.	Always an unsupervised technique.

Question 4: What are eigenvalues and eigenvectors in PCA, and why are they important?
Ans:- n **Principal Component Analysis (PCA)**, **eigenvalues** and **eigenvectors** are mathematical concepts used to identify the most important directions in the data.

Eigenvectors

- **Eigenvectors** represent the **directions (principal components)** along which the data varies the most.
- Each eigenvector defines a new axis in the transformed feature space.
- These axes are **orthogonal (perpendicular)** to each other, ensuring that the principal components are uncorrelated.

Eigenvalues

- **Eigenvalues** indicate the **amount of variance (information)** captured by each corresponding eigenvector.

- A **larger eigenvalue** means that the principal component explains more of the dataset's variation.
- A **smaller eigenvalue** means that the component contributes less information.

Importance in PCA

1. **Identify the most important components**
    - PCA selects the principal components with the **highest eigenvalues** because they retain the maximum information.
2. **Reduce dimensionality**
    - Components with very small eigenvalues can be discarded with minimal loss of information, reducing the number of features.
3. **Improve model performance**
    - Using only the most informative principal components reduces noise, speeds up computation, and can improve the performance of machine learning algorithms.

Question 5: How do KNN and PCA complement each other when applied in a single pipeline?

Ans:- **K-Nearest Neighbors (KNN)** and **Principal Component Analysis (PCA)** work well together because **PCA reduces the dimensionality of the data**, while **KNN uses the reduced data to make faster and more accurate predictions**.

How They Work Together

1. **Data Preprocessing**
    - Handle missing values.
    - Normalize or standardize the features.
2. **Apply PCA**

    - PCA transforms the original features into a smaller number of principal components.
    - These components retain most of the important information (variance) while removing noise and redundant features.
3. **Train KNN**
    - KNN uses the reduced dataset produced by PCA.
    - It calculates distances between data points in the lower-dimensional space and predicts the output.

Why PCA Improves KNN

- **Reduces the Curse of Dimensionality:** KNN performs better because distance calculations become more meaningful.
- **Faster Computation:** Fewer features mean fewer distance calculations, reducing prediction time.
- **Removes Noise:** PCA eliminates less important information, helping KNN focus on the most relevant patterns.
- **Improves Accuracy:** By reducing irrelevant and correlated features, KNN can make more reliable predictions.

Example Pipeline

1. Collect the dataset.
2. Clean the data and handle missing values.
3. Standardize the features.
4. Apply PCA to reduce the number of features (e.g., from 100 to 20).
5. Train the KNN model on the PCA-transformed data.
6. Evaluate the model using metrics such as **accuracy**, **precision**, **recall**, or **RMSE** (for regression).

Advantages of Combining PCA and KNN

- Improves prediction accuracy.
- Reduces computation time and memory usage.

- Handles high-dimensional datasets more effectively.
- Minimizes overfitting caused by irrelevant features.
- Makes KNN scalable to larger datasets.

Dataset: Use the Wine Dataset from sklearn.datasets.load_wine().
Question 6: Train a KNN Classifier on the Wine dataset with and without feature scaling. Compare model accuracy in both cases. (Include your Python code and output in the code box below.)

Ans:- # Import required libraries

```
from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score


# Load the Wine dataset

wine = load_wine()

X = wine.data

y = wine.target


# Split the dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42
```

```
)

# ----------------------------
# KNN without Feature Scaling
# ----------------------------
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)
accuracy_without_scaling = accuracy_score(y_test, y_pred)

print("Accuracy without Feature Scaling:",
      round(accuracy_without_scaling, 4))

# ----------------------------
# KNN with Feature Scaling
# ----------------------------
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)


knn_scaled = KNeighborsClassifier(n_neighbors=5)
knn_scaled.fit(X_train_scaled, y_train)


y_pred_scaled = knn_scaled.predict(X_test_scaled)
accuracy_with_scaling = accuracy_score(y_test, y_pred_scaled)


print("Accuracy with Feature Scaling:",
      round(accuracy_with_scaling, 4))
```

Output - Accuracy without Feature Scaling: 0.7222

Accuracy with Feature Scaling: 0.9722

Question 7: Train a PCA model on the Wine dataset and print the explained variance ratio of each principal component. (Include your Python code and output in the code box below.)

Ans:-

```
# Import required libraries
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA


# Load the Wine dataset
```

```
wine = load_wine()
X = wine.data

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train PCA
pca = PCA()
pca.fit(X_scaled)

# Print explained variance ratio
print("Explained Variance Ratio of each Principal Component:\n")

for i, ratio in enumerate(pca.explained_variance_ratio_, start=1):
    print(f"PC{i}: {ratio:.4f}")
```

Explained Variance Ratio of each Principal Component:

PC1: 0.3620

PC2: 0.1921

PC3: 0.1112

PC4: 0.0707

PC5: 0.0656

PC6: 0.0494

PC7: 0.0424

PC8: 0.0268

PC9: 0.0222

PC10: 0.0193

PC11: 0.0174

PC12: 0.0129

PC13: 0.0079

Question 8: Train a KNN Classifier on the PCA-transformed dataset (retain top 2 components). Compare the accuracy with the original dataset. (Include your Python code and output in the code box below.)

Ans:- # Import required libraries

```
from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score
```

```
# Load the Wine dataset
wine = load_wine()
X = wine.data
y = wine.target

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ----------------------------
# KNN on Original Dataset
# ----------------------------
knn_original = KNeighborsClassifier(n_neighbors=5)
```

```
knn_original.fit(X_train_scaled, y_train)

y_pred_original = knn_original.predict(X_test_scaled)
original_accuracy = accuracy_score(y_test, y_pred_original)

# ----------------------------
# Apply PCA (Top 2 Components)
# ----------------------------
pca = PCA(n_components=2)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# KNN on PCA-transformed Dataset
knn_pca = KNeighborsClassifier(n_neighbors=5)
knn_pca.fit(X_train_pca, y_train)

y_pred_pca = knn_pca.predict(X_test_pca)
pca_accuracy = accuracy_score(y_test, y_pred_pca)
```

Print Results

print("Accuracy on Original Dataset :", round(original_accuracy, 4))

print("Accuracy on PCA Dataset (2 Components):",
round(pca_accuracy, 4))

Accuracy on Original Dataset : 0.9722

Accuracy on PCA Dataset (2 Components): 0.9444

Question 9: Train a KNN Classifier with different distance metrics (euclidean, manhattan) on the scaled Wine dataset and compare the results. (Include your Python code and output in the code box below.)

Ans:- # Import required libraries

from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score

Load the Wine dataset

wine = load_wine()

X = wine.data

y = wine.target

```
# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ----------------------------
# KNN using Euclidean Distance
# ----------------------------
knn_euclidean = KNeighborsClassifier(
    n_neighbors=5,
    metric='euclidean'
)
knn_euclidean.fit(X_train_scaled, y_train)

y_pred_euclidean = knn_euclidean.predict(X_test_scaled)
```

```
accuracy_euclidean = accuracy_score(y_test, y_pred_euclidean)


# ----------------------------
# KNN using Manhattan Distance
# ----------------------------
knn_manhattan = KNeighborsClassifier(
    n_neighbors=5,
    metric='manhattan'
)
knn_manhattan.fit(X_train_scaled, y_train)


y_pred_manhattan = knn_manhattan.predict(X_test_scaled)
accuracy_manhattan = accuracy_score(y_test, y_pred_manhattan)


# Print Results
print("Accuracy using Euclidean Distance :",
round(accuracy_euclidean, 4))
print("Accuracy using Manhattan Distance:",
round(accuracy_manhattan, 4))
```

Output -

```
Accuracy using Euclidean Distance : 0.9722
```

Accuracy using Manhattan Distance: 0.9722

Question 10: You are working with a high-dimensional gene expression dataset to classify patients with different types of cancer. Due to the large number of features and a small number of samples, traditional models overfit. Explain how you would:

● Use PCA to reduce dimensionality

● Decide how many components to keep

● Use KNN for classification post-dimensionality reduction

● Evaluate the model

● Justify this pipeline to your stakeholders as a robust solution for real-world biomedical data (Include your Python code and output in the code box below.)

Ans :- # Import required libraries

```
from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


# Load dataset (Wine dataset used as a demonstration)

data = load_wine()
```

```
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Apply PCA (retain 95% variance)
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# Train KNN
knn = KNeighborsClassifier(n_neighbors=5)
```

```
knn.fit(X_train_pca, y_train)

# Predictions
y_pred = knn.predict(X_test_pca)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", round(accuracy, 4))

# Number of Principal Components
print("Number of Principal Components:", pca.n_components_)

# Cross-validation
cv_scores = cross_val_score(knn, X_train_pca, y_train, cv=5)
print("Cross-validation Accuracy:", round(cv_scores.mean(), 4))

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
# Confusion Matrix

print("Confusion Matrix:")

print(confusion_matrix(y_test, y_pred))
```